

## Implementation Guidelines (V2)

1. **Full pipeline:** The system takes a news post (text + image) and classifies it into one of three classes using two signals fused together (Semantic + image forensics).

| Stage                | Component        | Input                   | Output                      | Trains                     |
|----------------------|------------------|-------------------------|-----------------------------|----------------------------|
| Stage 1              | FND-CLIP         | Text + Image            | semantic vector             | Try (Frozen)               |
| Stage 2.1            | DCT extraction   | Image file              | DCT map [3 × 224 × 224]     | Math only no need training |
| Stage 2.2            | ResNet18 encoder | DCT map                 | forensic vector             | Need to be train           |
| Fusion stage         | Cross-Attention  | Semantic v + forensic v | Fused vector — 1024 numbers | Need to be train           |
| Classification stage | MLP classifier   | Fused vector            | 3 class scores (logits)     | Need to be train           |

### 2. Construct the Dataset (DGM4, NewsCLIPpings) →

DGM4(Manipulated image samples), NewsCLIPpings →(Real samples and OOC samples)

- Find the smallest class size in the datasets
- Sample that exact number from every class
- Split each class separately: 70% train / 15% validation / 15% test (split per class so all splits are balanced too)
- Shuffle the training set before each epoch

### 3. FND-CLIP

- Load your FND-CLIP model
- Freeze ALL parameters
- Set FND-CLIP to evaluation mode before using it (before any forward pass)
- Wrap every FND-CLIP call, no need learning reading only (saves GPU memory)
- After the first any forward pass, print the output shape it is [batch\_size, 768].

Output → **v\_semantic**

#### 4. DCT

DCT converts the image from pixels to frequency patterns.

- Load image with OpenCV
- Resize to exactly  $224 \times 224$  pixels
- Convert color space: BGR  $\rightarrow$  YCbCr  $\rightarrow$  take Y channel only.
- Apply 2D DCT to Y channel only ( here artifacts are appear strong)
- Apply log scaling:  $\log(|\text{value}| + 0.00000001)$
- Normalize to  $[0, 1]$  using per-image min and max (per image, not globally across the whole dataset)
- Output tensor shape  $[1, 224, 224]$
- Compute all DCT maps ONCE and save to disk as .pt files (do not compute during the training loop).
- During training, load the saved .pt file for each sample.

#### 5. ResNet18 Forensic Encoder (Trains)

ResNet18 takes the DCT map and learns which frequency patterns indicate manipulation.

- Load ResNet18 with pretrained=True
- Change the first convolution layer to accept 1 channel
- Remove the last fully connected layer (the original classifier)
- Add at the end: Flatten  $\rightarrow$  Dropout(0.3)  $\rightarrow$  Linear(512, 768) (output must be 768 to match v\_semantic)

Output  $\rightarrow$  v\_forensic [768] is the rich vector  $\rightarrow$  enters cross-attention fusion

#### 6. Auxiliary Classifier (at training stage only)

After ResNet18 produces v\_forensic  $\rightarrow$  create an extra classifier, to forces the forensic encoder to learn independently.

- Add one layer: Linear(768, 3) from v\_forensic.
  - Produce aux\_logits [3] to compute auxiliary loss during training
  - At inference: use only main\_logits from MLP
- Input  $\rightarrow$  ( v\_forensic ) output  $\rightarrow$  (aux\_logits)**

#### 7. Cross-Attention Fusion (Trains)

v\_semantic and v\_forensic ask questions of each other. Both updated then concatenated.  
(both vectors must be same size before attention)

1. Project v\_semantic: Linear (768, 512)
2. Project v\_forensic: Linear (768, 512)
3. Add sequence dimension to both: shape becomes  $[B, 1, 512]$ .
4. Direction 1: Semantic asks Forensic: Q=semantic, K=forensic, V=forensic (semantic learns what forensic knows)

1. Direction 2: Forensic asks Semantic: Q=forensic, K=semantic, V=semantic (forensic learns semantic context)
2. Residual connection: add updated vector + original projected vector (if attention learns nothing, original signal is preserved)
3. LayerNorm after each residual.
4. Remove sequence dimension: shape back to [B, 512] (removes middle dimension)
5. Concatenate both updated vectors → output [B, 1024]

| Parameter            | Value                 | Why                                          |
|----------------------|-----------------------|----------------------------------------------|
| Projection dim       | 512                   | Reduces parameters while keeping information |
| Attention heads      | 8                     | $512 / 8 = 64$ per head — standard ratio     |
| Attention dropout    | 0.1                   | Light dropout on attention weights           |
| batch_first          | True                  | Input must be [Batch, Sequence, Dim]         |
| Residual + LayerNorm | after both directions | signal preservation                          |
| Output dim           | 1024                  | 512 semantic + 512 forensic                  |

Source: FCINet 2024 paper

## 8. MLP Classifier (Trains)

Takes fused vector [1024] and produces 3 raw scores called logits, one score per class.

| Layer                        | Input | Output | Activation | Dropout |
|------------------------------|-------|--------|------------|---------|
| Linear 1                     | 1024  | 512    | ReLU       | 0.5     |
| Linear 2                     | 512   | 256    | ReLU       | 0.3     |
| Linear 3 output (raw logits) | 256   | 3      | None       | —       |

- Output = raw logits [3] (Not probabilities)
- Do NOT add Softmax (CrossEntropyLoss applies softmax internally)
- To get prediction: take argmax of logits → index 0=Real, 1=Manipulated, 2=OOC
- To get probabilities at inference: apply softmax(logits) manually, for reporting confidence scores.

## 9. The Full Model

Connect all components. The forward pass must follow this exact order.

**Training: return both outputs:**

1. Receive: text tokens, image tensor, DCT map
2. FND-CLIP → v\_semantic [768] (frozen)
3. ResNet18 → v\_forensic [768] (trains)
4. Auxiliary Linear(768,3) → aux\_logits [3] (training only)
5. Cross-Attention(v\_semantic, v\_forensic) → fused [1024] (trains)
6. MLP(fused) → main\_logits [3] (trains)
7. Return main\_logits AND aux\_logits

### At inference: return only main output

8. Receive: text tokens, image tensor, DCT map
9. FND-CLIP → `_v_semantic` [768]
10. ResNet18 → `_v_forensic` [768]
11. Cross-Attention → fused [1024]
12. MLP → `_main_logits` [3]
13. Return `main_logits` (apply softmax for probability scores)

## 10. Loss Function

Balanced dataset, so apply regular Cross Entropy.

- Main loss = Cross Entropy Loss(`main_logits`, labels)
- Auxiliary loss = Cross Entropy Loss(`aux_logits`, labels)
- Total loss = `main_loss` +  $0.1 \times \text{aux\_loss}$  (0.1 keeps auxiliary loss small , only guides)
- Cross Entropy Loss applies softmax automatically.

### Training Configuration → Source: FCINet 2024

| Parameter         | Value                              |
|-------------------|------------------------------------|
| Optimizer         | Adam                               |
| Learning rate     | 1e-4                               |
| Weight decay      | 1e-4                               |
| Batch size        | 64                                 |
| Max epochs        | 50                                 |
| LR scheduler      | StepLR × 0.1 at epoch 30           |
| Early stopping    | 10 epochs patience on val F1-macro |
| Gradient clipping | <code>max_norm</code> = 1.0        |

## 11. Training Loop Logic

### For each epoch at training phase:

1. Call `model.train()` (enables Dropout and BatchNorm training behaviour)
2. For each batch: forward pass → compute total loss → zero gradients → backward → clip gradients → optimizer step
3. After all batches: call `scheduler.step()`

### Validation phase after training phase each epoch:

1. Call `model.eval()` (disables Dropout)
2. For each batch in validation set: forward pass inside `torch.no_grad()` → argmax of `main_logits` → predicted class
3. Collect all predictions and true labels
4. Compute F1-macro = `sklearn f1_score(all_labels, all_preds, average='macro')`
5. If F1-macro improved: save model weights, reset patience to 0

6. If not improved: add 1 to patience. If patience reaches 10: stop training
7. Call `model.train()` again before next epoch (switch back to train mode after validation)

## 12. Evaluation on Test set

Load the best saved checkpoint. Run on test set.

All metrics are performance scores, they are not probabilities.

- apply softmax to logits first to get probabilities for AUC-ROC)
- Run evaluation also on MMFakeBench without fine-tuning to tests generalization to unseen data distribution.

| Metric                   |
|--------------------------|
| Accuracy                 |
| Precision per class      |
| Recall per class         |
| F1-macro                 |
| AUC-ROC (number + curve) |
| Confusion matrix         |